

4 Lecture 4: Monte Carlo Learning

Introduction

Lecture 3 中的 Policy Iteration 和 Value Iteration 算法都是 model-based RL (MBRL) 方法，因为从式 (3.24) 可以看出他们需要已知环境模型 $p(r|s, a)$ 。然而实际中很可能无法获取 p ，因此一个简单的想法是通过采样进行估算。

Example 4.1. 考虑简单的掷硬币实验，投掷结果记为 X ，若为正则 $X = +1$ ，反之 $X = 0$ ，我们的目标是估计 $\mathbb{E}[X]$ 。
 ① *Model-based*: 当概率模型已知时，如 $p(X = 1) = p(X = 0) = 0.5$ ，根据定义有 $\mathbb{E}[X] = \sum_x x \cdot p(x) = 0.5$ 。
 ② *Model-free*: 根据大数定律 (Law of Large Numbers) $\mathbb{E}[X] \approx \bar{x} = \frac{1}{N} \sum_{j=1}^N x_j$ ，因此可以进行多次采样以逼近 $\mathbb{E}[X]$ 。

4.1 MC Basic

MC Basic – the Simplest MC-based RL Algorithm

MC Basic 是最简单的基于 Monte Carlo 估计的 RL 算法，其本质是将 Policy Iteration 的 policy evaluation $v_\pi(s) = \sum_a \pi(a|s)q_\pi(s, a)$ 中的 action value 的计算使用 Monte Carlo Estimation 替换成 model-free 的。回顾：

1. **Policy Evaluation:** $v_{\pi_k} = r_{\pi_k} + \gamma \mathbf{P}_{\pi_k} v_{\pi_k}$, $v_{\pi_k}(s) = \sum_a \pi_k(a|s) \left(\sum_{r, s'} p(r, s'|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_{\pi_k}(s') \right)$
 2. **Policy Improvement:** $\pi_{k+1}(s) \in \arg \max_\pi \sum_a \pi(a|s) \underbrace{\left(\sum_{r, s'} p(r, s'|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_{\pi_k}(s') \right)}_{q_{\pi_k}(s, a)}, s \in \mathcal{S}$
- $$\Rightarrow \pi_{k+1}(a | s) = \begin{cases} 1, & a = a_k^*(s) \\ 0, & a \neq a_k^*(s) \end{cases}, \quad a_k^*(s) = \arg \max_a q_{\pi_k}(s, a).$$

可以看到 Policy Evaluation 和 Policy Improvement 中都需要计算 action value $q_{\pi_k}(s, a)$ 。在概率模型 $p(r, s'|s, a)$ 未知时，就需要使用 Monte Carlo estimation 进行估计。回顾 action value 最原始的定义式：

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a] = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | S_t = s, A_t = a] \quad (4.1)$$

其中 G_t 为 discounted return。从 (s, a) 开始根据 policy π_k 进行 N 次模拟 (trajectory)，它们的 return 记为 $g^{(i)}(s, a)$ ：

$$g^{(i)}(s, a) = r_{t+1}^{(i)} + \gamma r_{t+2}^{(i)} + \gamma^2 r_{t+3}^{(i)} + \dots, \quad i = 1, 2, \dots, N \quad (4.2)$$

那么 $\{g^{(i)}(s, a)\}_{i=1}^N$ 就是 G_t 的样本 (samples，在 RL 中称为经验，experience)。根据 Monte Carlo estimation 有：

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a] \approx \frac{1}{N} \sum_{i=1}^N g^{(i)}(s, a). \quad (4.3)$$

Note 4.1. ① MC Basic 是建立在 model-based 算法 Policy Iteration 之上的 model-free 算法，区别在于对 action value q_{π_k} 的计算从基于概率模型计算变成了基于 MC 的估计。② 由于 Policy Iteration 中 Policy Improvement $\pi_{new}(s) = \arg \max_a Q_\pi(s, a)$ 直接需要 action value Q 因此直接估计；若先估计 V 再通过 MC 计算 Q 误差可能增大。③ MC basic 在理论上具有启发性，是别的算法的基石，但由于其效率较低，因此几乎没有应用。④ MC Basic 与 Policy Iteration 一样仍然是收敛的，因为理论上若有足够多的 experience 就可以足够精确地逼近 action value q_{π_k} 。

Note 4.2. Sparse Reward: 在使用 MC estimation 时 trajectory 的长度会对最终的估计产生影响，一个有趣的现象是 *sparse reward*，是指 reward 的设计很稀疏，可能只在达到 target 时才有 positive reward，因此对于一个 trajectory 的最短步数有要求，因为如果步数不够就没有能到达 target 的可能，也就不能获得 reward，这对学习效率有很大影响，因此解决此问题的一个方案是设计 *nonsparse rewards*，例如到达靠近 target 的区域也设置 positive reward。

MC Basic Algorithm

Algorithm 4 MC Basic Algorithm (a model-free variant of Policy Iteration)

```

1: Initialization: Initial guess  $\pi_0$ .
2: Aim: Search for an optimal policy.
3: while the value estimate has not converged, for the  $k$ -th iteration do
4:   for every state  $s \in \mathcal{S}$  do
5:     for every action  $a \in \mathcal{A}(s)$  do
6:       Collect sufficiently many trajectories starting from  $(s, a)$  following  $\pi_k$ .
7:       MC-based policy evaluation step:

$$q_{\pi_k}(s, a) \leftarrow \text{average return of all the trajectories starting from } (s, a)$$

8:     end for
9:     Policy improvement step:

$$a_k^*(s) \leftarrow \arg \max_a q_{\pi_k}(s, a)$$


$$\pi_{k+1}(a | s) = 1 \text{ if } a = a_k^*(s), \text{ and } \pi_{k+1}(a | s) = 0 \text{ otherwise}$$

10:   end for
11: end while

```

4.2 MC Exploring Starts

MC Exploring Starts – Use Data More Efficiently

在一条 trajectory 中，我们会连续访问许多 state-action pair，例如对于这样的一个人 trajectory:

$$s_1 \xrightarrow[r=0]{a_2} s_2 \xrightarrow[r=0]{a_4} s_1 \xrightarrow[r=0]{a_2} s_2 \xrightarrow[r=1]{a_3} s_5 \xrightarrow[r=1]{a_1} \dots \quad (4.4)$$

MC Basic 算法使用的策略是 **Initial-visit method**，即原理上使用一整个 trajectory 计算出发点处 state-action pair (s_1, a_2) 的 action value $q_\pi(s_1, a_2)$ ，但其缺点是一个 trajectory 的数据没有被充分利用，因为一个完整的 trajectory 事实上可以依次拆分成如下多个 sub-trajectories:

$$\begin{aligned}
& s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots [\text{original trajectory, estimate } q_\pi(s_1, a_2)] \\
& s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots [\text{trajectory starting from } (s_2, a_4), \text{ estimate } q_\pi(s_2, a_4)] \\
& s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots [\text{trajectory starting from } (s_1, a_2), \text{ estimate } q_\pi(s_1, a_2)] \\
& s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots [\text{trajectory starting from } (s_2, a_3), \text{ estimate } q_\pi(s_2, a_3)] \\
& s_5 \xrightarrow{a_1} \dots [\text{trajectory starting from } (s_5, a_1), \text{ estimate } q_\pi(s_5, a_1)]
\end{aligned} \quad (4.5)$$

First vs. Every. 为了能够更充分利用一个 trajectory 的数据，更 data-efficient 的方法是计算其中所有出现过 visit (state-action pair) 的 action value. 同时，在一个 trajectory 中同一个 state-action pair 可能出现多次，这就产生了两种方式: ① **First-visit Method:** 只使用第一次出现时计算得到的 $q_\pi(s_i, a_j)$ 作为返回值，第二次的计算. 这样对于一个 state-action pair，一个 trajectory 只能得到一次采样. ② **Every-visit Method:** 对于重复出现的 state-action pair，每一次都计算 action value. 这样对于一个 state-action pair，一个 trajectory 可能得到多次采样.

Efficiency vs. Accuracy. 在 MC-based RL 中另一个重要的问题是什么时候更新 policy，这里仍然有两种方式: ① **Average return:** 在 MC Basic 中，使用多个 trajectory 取平均的 average return 作为 action value 的估计值，这样效率较慢但是一次得到的 action value 较准确. ② **Single trajectory:** 只使用一次 trajectory 来估计 action value，这样虽然可能不准确，但更新效率更高. 这种思想与 truncated policy iteration 类似.

MC Exploring Starts Algorithm

MC Exploring Starts 中 *exploring* 指我们需要从一个 state-action pair 出发，访问到所有的 state-action pair 才能知道某一个 action 是否为最优，因此需要尽量 explore 所有的 pair. Starts 是指从一个 visit 出发有限次访问可能访问不到某些 pair，因此一般需要随机从一个 start 开始 explore，保证遍历所有的 state-action pair。算法如下：

Algorithm 5 MC Exploring Starts (a sample-efficient variant of MC Basic)

```

1: Initialization: Initial guess  $\pi_0$ .
2: Aim: Search for an optimal policy.
3: trajectory generation: Randomly select a starting state-action pair  $(s_0, a_0)$  and ensure that all pairs can be possibly selected.  $\triangleright$  exploring starts requirement
4: for each trajectory do
5:   Following the current policy, generate a trajectory of length  $T$ :  $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$ .
6:   Policy evaluation and policy improvement:
7:     Initialization:  $g \leftarrow 0$ 
8:     for each step of the trajectory,  $t = T-1, T-2, \dots, 0$  do  $\triangleright$  compute inversely to improve efficiency
9:        $g \leftarrow \gamma g + r_{t+1}$ 
10:       $\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$ 
11:       $\text{Num}(s_t, a_t) \leftarrow \text{Num}(s_t, a_t) + 1$ 
12:      Policy evaluation:  $q(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) / \text{Num}(s_t, a_t)$ 
13:      Policy improvement:  $\begin{cases} 1 & \text{if } a = \arg \max_a q(s_t, a), \\ 0 & \text{otherwise.} \end{cases}$ 
14:     end for
15: end for

```

Note 4.3. ① 形式上，如果起始分布满足 $P(S_0 = s, A_0 = a) > 0$ 对所有 state-action pair 都成立，那么经过足够多 trajectory 后，每个 (s, a) 都会被反复采样。② Exploring Starts 是一种理论上很方便的探索假设，但在物理世界中通常难以实现，因为可能无法把机器人直接放到任意状态并强制执行任意动作。③ 在计算一个 trajectory 中正向计算 g_t 需要预先知晓后续的 g_{t+1} : $g_t = \gamma g_{t+1} + r_{t+1}$ ，因此采用反向计算，先计算后续的 g_t 以提高计算效率。④ 显然只有当一个 state-action pair 被充分访问多次时才能准确估计其 action value，这被称为 *exploring starts requirement*。

4.3 MC ϵ -Greedy

MC ϵ -Greedy — MC without Exploring Starts

在实际中由于物理限制很难对 start 进行 explore，因此更常见的做法不是强制改变初始状态动作，而是在访问状态时给所有动作一定探索概率。此时只需一个足够长的 trajectory 也可以访问每一个 state-action pair 很多次，满足 exploring starts requirement。这样的 policy 被称为 **soft policy**，即在任意 state 任意 action 都有概率被执行。

MC ϵ -Greedy 的核心就是在 policy update 时将 policy 变成 soft policy 以去除 exploring starts requirement:

$$\pi(a | s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1), & \text{for the greedy action} \\ \frac{\epsilon}{|\mathcal{A}(s)|}, & \text{for the other } |\mathcal{A}(s)| - 1 \text{ actions} \end{cases} \quad (4.6)$$

其中 $\epsilon \in [0, 1]$ ， $\mathcal{A}(s)$ 为 state s 对应的 action space。

Note 4.4. 可以看到 soft policy / ϵ -greedy policy 仍将最大的概率保留给了 action value 最大的 greedy action:

$$1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|} \geq \frac{\epsilon}{|\mathcal{A}(s)|}. \quad (4.7)$$

Exploitation vs. Exploration. 使用 ϵ -greedy policy 的原因是平衡 **exploitation** (始终选择最优) 与 **exploration** (探索)。Exploitation 指只选择最优 action (即 greedy)，exploration 给别的 action 也分配机会，防止陷入局部最优。

Note 4.5. ① 当 $\epsilon = 0$ 时，就变成了 greedy，此时 *less exploration but more exploitation*。② 当 $\epsilon = 1$ 时，所有的 action 选择概率相同，此时 *more exploration*。③ 在实际使用中，可以设置较小的 ϵ ，然后再逐渐减小。

Implementation. 在实际算法部署中可以通过 $[0, 1]$ 上的均匀分布产生一随机数 x ，若 $x \geq \epsilon$ 则选择 greedy action，否则对于 $\mathcal{A}(s)$ 中所有的 action 都有 $\frac{1}{|\mathcal{A}(s)|}$ 的概率选择到。

Consistency: 在 ϵ -Greedy 中，如果每一个 state 采取的动作都有一定的概率，但是最大概率对应的 action 与 optimal action 一致，那么就称此 ϵ 下的算法是 consistent 的。一般来说当 ϵ 较小时更容易保证 consistent。

MC ϵ -Greedy Algorithm

将 ϵ -greedy 的策略替换掉原先的 greedy 的策略，就可得到 MC ϵ -Greedy algorithm，同时注意此时 trajectory generation 时 start 的选择需要符合环境规定。

Algorithm 6 MC ϵ -Greedy (a variant of MC Exploring Starts)

```
1: Initialization: Initial guess  $\pi_0$  and the value of  $\epsilon \in [0, 1]$ .
2: Aim: Search for an optimal policy.
3: trajectory generation: Randomly select an environment-permitted starting state-action pair  $(s_0, a_0)$ . Following the current policy, generate an trajectory of length  $T$ :  $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$ .
4: for each trajectory do
5:   Policy evaluation and policy improvement:
6:     Initialization:  $g \leftarrow 0$ 
7:     for each step of the trajectory,  $t = T - 1, T - 2, \dots, 0$  do ▷ for efficiency
8:        $g \leftarrow \gamma g + r_{t+1}$ 
9:        $\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$ 
10:       $q(s_t, a_t) \leftarrow \text{average}(\text{Returns}(s_t, a_t))$ 
11:      Let  $a^* = \arg \max_a q(s_t, a)$  and
12:       $\pi(a \mid s_t) \leftarrow \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s_t)|} & \text{if } a = a^*, \\ \frac{\epsilon}{|\mathcal{A}(s_t)|} & \text{if } a \neq a^*. \end{cases}$ 
13:    end for
14: end for
```
